

APOLLO INSTITUTE OF COMPUTER APPLICATION	
BACHELOR OF COMPUTER APPLICATION	
SEMESTER 03	
Object Oriented Programming with Java (BC03001021)	
Lab Manual	
Lab No.	Practical (As per GTU Syllabus)
1	Write a program in Java to generate first n prime numbers.
2	Write a program in Java to find maximum of three numbers using conditional operator.
3	Write a program in Java to find second maximum of n numbers without using arrays.
4	Write a program in Java to reverse the digits of a number using while loop.
5	Write a program in Java to convert number into words & print it.
6	Write programs in Java to use Wrapper class of each primitive data types.
7	Write a program in Java to multiply two matrix.
8	Write a static block which will be executed before main() method in a class.
9	Write a program in Java to demonstrate use of this keyword. Check whether this can access the private members of the class or not.
10	Write a program in Java to develop an overloaded constructor. Also develop the copy constructor to create a new object with the state of the existing object.
11	Write a program in Java to demonstrate the use of private constructor and also write a method which will count the number of instances created using default constructor only.
12	Write a program in Java to demonstrate the use of 'final' keyword in the field declaration. How it is accessed using the objects.
13	Develop minimum 4 program based on variation in methods i.e. passing by value, passing by reference, returning values and returning objects from methods.
14	Write a program in Java to demonstrate single inheritance, multilevel inheritance and hierarchical inheritance.
15	Create a class to find out whether the given year is leap year or not. (Use inheritance for this program).

16	Write an application that illustrates how to access a hidden variable. Class A declares a static variable x. The class B extends A and declares an instance variable x. display() method in B displays both of these variables.
17	Write a program in Java in which a subclass constructor invokes the constructor of the super class and instantiate the values.
18	Write a program that illustrates interface inheritance. Interface P12 inherits from both P1 and P2. Each interface declares one constant and one method. The class Q implements P12. Instantiate Q and invoke each of its methods. Each method displays one of the constants.
19	Write an application that illustrates method overriding in the same package and different packages. Also demonstrate accessibility rules inside and outside packages.
20	Describe abstract class called Shape which has three subclasses say Triangle, Rectangle, Circle. Define one method area() in the abstract class and override this area() in these three subclasses to calculate the area.
21	Write a program in Java to demonstrate implementation of multiple inheritance using interfaces.
22	Write a program in Java to demonstrate use of final class.
23	Write a program in Java to develop user defined exception for 'Divide by Zero' error.
24	Write a program in Java to demonstrate multiple try block and multiple catch exception.
25	Write a small application in Java to develop Banking Application in which user deposits Rs.1000.00 and withdraws Rs.400.00, Rs.300.00 then throws 'Not Sufficient Fund' on Rs.500 withdrawal.
26	Write a program that executes two threads. One thread displays "Thread1" every 2,000 milliseconds, and the other displays "Thread2" every 4,000 milliseconds. Create the threads by extending the Thread class.
27	Write a program that executes two threads. One thread will print the even numbers and the other thread will print odd numbers from 1 to 50.
28	Write a program in Java to demonstrate use of synchronization of threads when multiple threads are trying to update common variable.
29	Write a program in Java to create, write, modify, read operations on a Text file.
30	Viva, Practical Evaluation and Mini Project Demonstration

Experiment No. 1

Aim

Write a Java program to generate the first **n** prime numbers.

Theory

A **prime number** is a positive integer greater than 1 that has exactly two factors: **1** and **itself**. Examples are 2, 3, 5, 7, 11, etc.

Algorithm

1. Start.
 2. Read the value of **n**.
 3. Initialize count = 0 and num = 2.
 4. Check whether num is prime.
 5. If prime, print it and increment count.
 6. Repeat until count == n.
 7. Stop.
-

Program

```
import java.util.Scanner;

public class PrimeNumbers {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n;
        System.out.print("Enter value of n: ");
        n = sc.nextInt();

        int count = 0;
        int num = 2;

        System.out.println("First " + n + " Prime Numbers are:");

        while (count < n) {

            boolean prime = true;

            for (int i = 2; i <= num / 2; i++) {
```

```

        if (num % i == 0) {
            prime = false;
            break;
        }
    }

    if (prime) {
        System.out.print(num + " ");
        count++;
    }

    num++;
}

sc.close();
}
}

```

Line-by-Line Explanation

1. Import Scanner Class

```
import java.util.Scanner;
```

- Imports the **Scanner** class from the java.util package.
 - It is used to take input from the keyboard.
-

2. Create Class

```
public class PrimeNumbers {
```

- Declares a class named **PrimeNumbers**.
 - Every Java program is written inside a class.
-

3. Main Method

```
public static void main(String[] args) {
```

- The main() method is the entry point of the Java program.
 - Program execution starts from this method.
-

4. Create Scanner Object

```
Scanner sc = new Scanner(System.in);
```

- Creates an object of the Scanner class.
 - System.in means input will be taken from the keyboard.
-

5. Declare Variable

```
int n;
```

- Declares an integer variable n.
 - It stores how many prime numbers the user wants.
-

6. Ask User for Input

```
System.out.print("Enter value of n: ");
```

```
n = sc.nextInt();
```

- Displays a message.
- Reads an integer value entered by the user.

Example

Enter value of n: 10

Now,

```
n = 10
```

7. Initialize Variables

```
int count = 0;
```

```
int num = 2;
```

- count stores how many prime numbers have been printed.
- Initially, no prime number has been printed.

```
count = 0
```

- num starts from **2**, because 2 is the first prime number.

```
num = 2
```

8. Display Heading

```
System.out.println("First " + n + " Prime Numbers are:");
```

Output

First 10 Prime Numbers are:

9. While Loop

```
while (count < n)
```

The loop continues until the required number of prime numbers is printed.

Example

```
count = 0
```

$n = 10$

$0 < 10$ ✓

Loop starts.

10. Assume Number is Prime

boolean prime = true;

Initially every number is assumed to be prime.

For example

num = 7

prime = true

11. Check Divisibility

for (int i = 2; i <= num / 2; i++)

Checks whether num is divisible by any number from **2** to **num/2**.

Example

num = 9

$\text{num}/2 = 4$

Loop checks

$9 \div 2$

$9 \div 3$

$9 \div 4$

12. If Divisible

if (num % i == 0)

% gives the remainder.

Example

$9 \% 3 = 0$

Since remainder is 0,

9 is **not prime**.

13. Mark as Not Prime

prime = false;

Changes the flag.

prime = false

14. Exit Loop

break;

No need to check further because the number is already found to be non-prime.

15. Print Prime Number

if (prime)

If the flag is still true, the number is prime.

Example

num = 13

prime = true

Print

13

16. Print Number

System.out.print(num + " ");

Displays the prime number.

Example

2 3 5 7 11

17. Increase Count

count++;

Increases the number of prime numbers printed.

Example

Before

count = 4

After

count = 5

18. Check Next Number

num++;

Moves to the next number.

Example

2 → 3 → 4 → 5 → 6 → 7 ...

19. Close Scanner

sc.close();

Sample Output

Enter value of n: 10

First 10 Prime Numbers are:

2 3 5 7 11 13 17 19 23 29

Experiment No. 2

Aim

Write a Java program to find the maximum of three numbers using the **conditional (ternary) operator**.

Theory

The conditional operator (?:) is a shorthand for the if-else statement. It returns one of two values depending on a condition.

Algorithm

1. Read three numbers.
2. Compare them using nested conditional operators.

3. Display the largest number.

Program

```
import java.util.Scanner;

public class MaximumThree {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int a, b, c, max;

        System.out.print("Enter First Number: ");
        a = sc.nextInt();

        System.out.print("Enter Second Number: ");
        b = sc.nextInt();

        System.out.print("Enter Third Number: ");
        c = sc.nextInt();

        max = (a > b) ? ((a > c) ? a : c)
              : ((b > c) ? b : c);

        System.out.println("Maximum Number = " + max);

        sc.close();
    }
}
```

Sample Output

```
Enter First Number: 25
Enter Second Number: 78
Enter Third Number: 45
```

```
Maximum Number = 78
```

Experiment No. 3

Aim

Write a Java program to find the **second maximum** of **n numbers** without using arrays.

Theory

The program maintains two variables to store the largest and second largest numbers while taking input one by one.

Algorithm

1. Read n.
 2. Read first number as maximum.
 3. Read remaining numbers.
 4. Update first and second maximum.
 5. Display second maximum.
-

Program

```
import java.util.Scanner;

public class SecondMaximum {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n;

        System.out.print("Enter Number of Elements: ");
        n = sc.nextInt();

        int first = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;

        System.out.println("Enter Numbers:");

        for (int i = 1; i <= n; i++) {

            int num = sc.nextInt();

            if (num > first) {
```

```
        second = first;
        first = num;
    } else if (num > second && num != first) {
        second = num;
    }
}

System.out.println("Second Maximum = " + second);

    sc.close();
}
}
```

Sample Output

Enter Number of Elements: 5

12
85
45
67
29

Second Maximum = 67

Experiment No. 4

Aim

Write a Java program to reverse the digits of a number using a **while loop**.

Theory

The reverse of a number is obtained by repeatedly extracting the last digit using the modulus operator (%) and constructing the reversed number.

Algorithm

1. Read a number.
2. Extract last digit.
3. Append digit to reverse.

4. Remove last digit.
 5. Repeat until number becomes zero.
-

Program

```
import java.util.Scanner;

public class ReverseNumber {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int num, reverse = 0;

        System.out.print("Enter Number: ");
        num = sc.nextInt();

        while (num != 0) {

            int digit = num % 10;
            reverse = reverse * 10 + digit;
            num = num / 10;

        }

        System.out.println("Reverse Number = " + reverse);

        sc.close();
    }
}
```

Sample Output

Enter Number: 12345

Reverse Number = 54321

Experiment No. 5

Aim

Write a Java program to convert a number into words and print it.

Theory

Each digit of the number is converted into its corresponding word using a switch statement.

Algorithm

1. Read the number.
 2. Reverse the number.
 3. Read each digit.
 4. Convert the digit into words.
 5. Display the words.
-

Program

```
import java.util.Scanner;

public class NumberToWords {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int num, rev = 0;

        System.out.print("Enter Number: ");
        num = sc.nextInt();

        int temp = num;

        while (temp != 0) {
            rev = rev * 10 + temp % 10;
            temp = temp / 10;
        }

        while (rev != 0) {

            int digit = rev % 10;
```

```
switch (digit) {
    case 0:
        System.out.print("Zero ");
        break;
    case 1:
        System.out.print("One ");
        break;
    case 2:
        System.out.print("Two ");
        break;
    case 3:
        System.out.print("Three ");
        break;
    case 4:
        System.out.print("Four ");
        break;
    case 5:
        System.out.print("Five ");
        break;
    case 6:
        System.out.print("Six ");
        break;
    case 7:
        System.out.print("Seven ");
        break;
    case 8:
        System.out.print("Eight ");
        break;
    case 9:
        System.out.print("Nine ");
        break;
}

    rev = rev / 10;
}

sc.close();
}
```

Sample Output

Enter Number: 2589

Two Five Eight Nine

Experiment No. 6

Aim

Write a Java program to demonstrate the use of **Wrapper Classes** of each primitive data type.

Theory

Java provides **Wrapper Classes** for all primitive data types. A wrapper class converts a primitive value into an object (Autoboxing) and converts an object back to a primitive value (Unboxing).

Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Algorithm

1. Declare variables of all primitive data types.
 2. Convert primitive values into wrapper objects.
 3. Display wrapper objects.
 4. Convert wrapper objects back into primitive values.
 5. Display primitive values.
-

Program

```
public class WrapperDemo {  
  
    public static void main(String[] args) {  
  
        byte b = 10;  
        short s = 20;  
        int i = 100;  
        long l = 1000;  
        float f = 12.5f;  
        double d = 25.75;
```

```
char c = 'A';
boolean flag = true;

// Autoboxing
Byte bObj = b;
Short sObj = s;
Integer iObj = i;
Long lObj = l;
Float fObj = f;
Double dObj = d;
Character cObj = c;
Boolean boolObj = flag;
```

```
System.out.println("Wrapper Objects:");
System.out.println(bObj);
System.out.println(sObj);
System.out.println(iObj);
System.out.println(lObj);
System.out.println(fObj);
System.out.println(dObj);
System.out.println(cObj);
System.out.println(boolObj);
```

```
// Unboxing
int number = iObj;
System.out.println("\nPrimitive Value = " + number);
```

```
}
```

```
}
```

Sample Output

Wrapper Objects:

10

20

100
1000
12.5
25.75
A
true

Primitive Value = 100

Experiment No. 7

Aim

Write a Java program to multiply two matrices.

Theory

Matrix multiplication is performed by multiplying the rows of the first matrix with the columns of the second matrix.

Algorithm

1. Read the size of matrices.
2. Read Matrix A.
3. Read Matrix B.
4. Multiply matrices.
5. Display the resultant matrix.

Program

```
import java.util.Scanner;

public class MatrixMultiplication {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int[][] A = new int[2][2];
```

```
int[][] B = new int[2][2];
```

```
int[][] C = new int[2][2];
```

```
System.out.println("Enter First Matrix:");
```

```
for(int i=0;i<2;i++)
```

```
    for(int j=0;j<2;j++)
```

```
        A[i][j]=sc.nextInt();
```

```
System.out.println("Enter Second Matrix:");
```

```
for(int i=0;i<2;i++)
```

```
    for(int j=0;j<2;j++)
```

```
        B[i][j]=sc.nextInt();
```

```
for(int i=0;i<2;i++)
```

```
{
```

```
    for(int j=0;j<2;j++)
```

```
    {
```

```
        C[i][j]=0;
```

```
        for(int k=0;k<2;k++)
```

```
        {
```

```
            C[i][j]+=A[i][k]*B[k][j];
```

```
        }
```

```
    }
```

```
}
```

```
System.out.println("Resultant Matrix:");
```

```
for(int i=0;i<2;i++)
```

```
{
```

```
    for(int j=0;j<2;j++)
```

```
        {  
            System.out.print(C[i][j]+" ");  
        }  
        System.out.println();  
    }  
  
    sc.close();  
}  
}
```

Sample Output

Enter First Matrix

1 2
3 4

Enter Second Matrix

5 6
7 8

Resultant Matrix

19 22
43 50

Experiment No. 8

Aim

Write a Java program to demonstrate that a **static block** executes before the `main()` method.

Theory

A static block is executed automatically when the class is loaded into memory before the execution of the `main()` method.

Algorithm

1. Create a static block.
2. Write the main method.
3. Execute the program.
4. Observe the execution order.

Program

```
public class StaticBlockDemo {  
  
    static {  
  
        System.out.println("Static Block Executed");  
  
    }  
  
    public static void main(String[] args) {  
  
        System.out.println("Main Method Executed");  
  
    }  
}
```

Sample Output

```
Static Block Executed  
Main Method Executed
```

Experiment No. 9

Aim

Write a Java program to demonstrate the use of the **this** keyword and verify whether it can access private members of the class.

Theory

The **this** keyword refers to the current object. It is used to access instance variables, invoke methods, and call constructors. It can also access private members of the same class.

Algorithm

1. Create a private data member.
 2. Initialize it using the this keyword.
 3. Display the value.
 4. Execute the program.
-

Program

```
class Student {  
  
    private int rollNo;  
  
    Student(int rollNo) {  
  
        this.rollNo = rollNo;  
  
    }  
  
    void display() {  
  
        System.out.println("Roll Number = " + this.rollNo);  
  
    }  
}  
  
public class ThisDemo {  
  
    public static void main(String args[]) {  
  
        Student s = new Student(101);
```

```
s.display();  
  
}  
}
```

Sample Output

Roll Number = 101

Experiment No. 10

Aim

Write a Java program to demonstrate **constructor overloading** and **copy constructor**.

Theory

A constructor initializes an object. Java supports constructor overloading by defining multiple constructors with different parameter lists. A copy constructor creates a new object by copying the values of another object.

Algorithm

1. Create a default constructor.
 2. Create a parameterized constructor.
 3. Create a copy constructor.
 4. Create objects using all constructors.
 5. Display object details.
-

Program

```
class Student {  
  
    int id;  
    String name;  
  
    Student() {  
  
        id = 0;  
        name = "Unknown";  
    }  
}
```

```
}
```

```
Student(int id, String name) {
```

```
    this.id = id;
```

```
    this.name = name;
```

```
}
```

```
Student(Student s) {
```

```
    id = s.id;
```

```
    name = s.name;
```

```
}
```

```
void display() {
```

```
    System.out.println("ID : " + id);
```

```
    System.out.println("Name : " + name);
```

```
}
```

```
}
```

```
public class ConstructorDemo {
```

```
    public static void main(String args[]) {
```

```
        Student s1 = new Student();
```

```
        Student s2 = new Student(101, "Rahul");
```

```
        Student s3 = new Student(s2);
```

```
        System.out.println("Default Constructor");

        s1.display();

        System.out.println();

        System.out.println("Parameterized Constructor");

        s2.display();

        System.out.println();

        System.out.println("Copy Constructor");

        s3.display();

    }
}
```

Sample Output

Default Constructor

ID : 0

Name : Unknown

Parameterized Constructor

ID : 101

Name : Rahul

Copy Constructor

ID : 101

Name : Rahul

Experiment No. 11

Aim

Write a Java program to demonstrate the use of a **private constructor** and write a method to count the number of instances created using the **default constructor only**.

Theory

- A **private constructor** prevents objects from being created outside the class.
- A **static counter** is used to count objects created using the default constructor.

Algorithm

1. Declare a static counter variable.
2. Create a default constructor to increment the counter.
3. Create a private constructor.
4. Display the total number of objects created.

Program

```
class Demo {  
  
    static int count = 0;  
  
    Demo() {  
        count++;  
    }  
  
    private Demo(int x) {  
        System.out.println("Private Constructor Called");  
    }  
  
    static void displayCount() {  
        System.out.println("Objects created using default constructor = " + count);  
    }  
  
    public static void main(String args[]) {  
  
        Demo d1 = new Demo();  
        Demo d2 = new Demo();  
        Demo d3 = new Demo();  
    }  
}
```

```
displayCount();
```

```
// Demo d4 = new Demo(10); // Not allowed outside the class
```

```
}
```

```
}
```

Sample Output

Objects created using default constructor = 3

Experiment No. 12

Aim

Write a Java program to demonstrate the use of the **final** keyword in field declaration.

Program

```
class Student {
```

```
    final int ID = 101;
```

```
    void display() {
```

```
        System.out.println("Student ID = " + ID);
```

```
    }
```

```
}
```

```
public class FinalDemo {
```

```
    public static void main(String args[]) {
```

```
        Student s = new Student();
```

```
        s.display();
```

```
        // s.ID = 200; // Error
```

```
    }
```

```
}
```

Sample Output

Student ID = 101

Experiment No. 13

Aim

Develop four Java programs demonstrating:

- Passing by Value
- Passing by Reference
- Returning Value
- Returning Object

Program 1 – Passing by Value

```
class PassValue {  
  
    void change(int x) {  
        x = x + 10;  
    }  
  
    public static void main(String args[]) {  
  
        int a = 20;  
  
        PassValue obj = new PassValue();  
  
        obj.change(a);  
  
        System.out.println(a);  
  
    }  
}
```

Output

20

Program 2 – Passing by Reference

```
class Test {  
  
    int value = 20;  
}  
  
public class PassReference {  
  
    void change(Test t) {  
        t.value = 50;  
    }  
  
    public static void main(String args[]) {  
  
        Test t = new Test();  
  
        PassReference obj = new PassReference();  
  
        obj.change(t);  
  
        System.out.println(t.value);  
  
    }  
}
```

Output

50

Program 3 – Returning Value

```
public class ReturnValue {  
  
    int square(int n) {  
        return n * n;  
    }  
}
```

```
public static void main(String args[]) {  
  
    ReturnValue obj = new ReturnValue();  
  
    System.out.println(obj.square(5));  
  
}  
}
```

Output

25

Program 4 – Returning Object

```
class Student {  
  
    int id = 101;  
  
    Student getObject() {  
        return this;  
    }  
  
    void display() {  
        System.out.println(id);  
    }  
}  
  
public class ReturnObject {  
  
    public static void main(String args[]) {  
  
        Student s = new Student();  
  
        Student s2 = s.getObject();  

```

```
s2.display();
```

```
}
```

```
}
```

Output

101

Experiment No. 14

Aim

Write Java programs demonstrating:

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance

Program

(Separate classes for each inheritance type.)

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking...");  
    }  
}
```

```
public class SingleInheritance {  
    public static void main(String args[]) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

```
}  
}
```

Output

Eating...

Barking...

Experiment No. 15

Aim

Create a class to find whether a given year is a leap year using inheritance.

Program

```
import java.util.Scanner;
```

```
class Year {
```

```
    int year;
```

```
    void getYear() {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter Year: ");
```

```
        year = sc.nextInt();
```

```
    }
```

```
}
```

```
class LeapYear extends Year {
```

```
    void check() {
```

```
        if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
```

```
            System.out.println("Leap Year");
```

```
else
```

```
    System.out.println("Not Leap Year");
```

```
}
```

```
}
```

```
public class LeapDemo {
```

```
    public static void main(String args[]) {
```

```
        LeapYear obj = new LeapYear();
```

```
        obj.getYear();
```

```
        obj.check();
```

```
    }
```

```
}
```

Output

Enter Year: 2024

Leap Year

Experiment No. 16

Aim

Illustrate how to access a hidden variable.

Program

```
class A {
```

```
    static int x = 10;
```

```
}
```

```
class B extends A {  
  
    int x = 20;  
  
    void display() {  
  
        System.out.println("Instance x = " + x);  
  
        System.out.println("Static x = " + A.x);  
  
    }  
}
```

```
public class HiddenVariable {  
  
    public static void main(String args[]) {  
  
        B obj = new B();  
  
        obj.display();  
  
    }  
}
```

Output

Instance x = 20

Static x = 10

Experiment No. 17

Aim

Write a Java program in which a subclass constructor invokes the constructor of the superclass.

Program

```
class Person {  
  
    Person() {  
  
        System.out.println("Person Constructor");  
  
    }  
}  
  
class Student extends Person {  
  
    Student() {  
  
        super();  
  
        System.out.println("Student Constructor");  
  
    }  
}  
  
public class SuperDemo {  
  
    public static void main(String args[]) {  
  
        Student s = new Student();  
  
    }  
}
```

Output

Person Constructor

Student Constructor

Experiment No. 18

Aim

Write a Java program to illustrate interface inheritance.

Program

```
interface P1 {

    int A = 10;

    void show1();
}

interface P2 {

    int B = 20;

    void show2();
}

interface P12 extends P1, P2 {

}

class Q implements P12 {

    public void show1() {

        System.out.println("A = " + A);

    }

    public void show2() {
```

```
        System.out.println("B = " + B);

    }

}

public class InterfaceDemo {

    public static void main(String args[]) {

        Q obj = new Q();

        obj.show1();

        obj.show2();

    }

}
```

Output

A = 10

B = 20

Experiment No. 19

Aim

Write a Java program to demonstrate method overriding.

Program

```
class Animal {

    void sound() {

        System.out.println("Animal Sound");

    }

}
```

```
}
```

```
class Dog extends Animal {
```

```
    @Override
```

```
    void sound() {
```

```
        System.out.println("Dog Barks");
```

```
    }
```

```
}
```

```
public class OverrideDemo {
```

```
    public static void main(String args[]) {
```

```
        Animal a = new Dog();
```

```
        a.sound();
```

```
    }
```

```
}
```

Output

Dog Barks

Experiment No. 20

Aim

Create an abstract class **Shape** with subclasses **Triangle**, **Rectangle**, and **Circle**. Override the `area()` method in each subclass.

Program

```
abstract class Shape {
```

```
    abstract void area();
```

```
}
```

```
class Triangle extends Shape {
```

```
    void area() {
```

```
        System.out.println("Area of Triangle = " + (0.5 * 10 * 5));
```

```
    }
```

```
}
```

```
class Rectangle extends Shape {
```

```
    void area() {
```

```
        System.out.println("Area of Rectangle = " + (10 * 5));
```

```
    }
```

```
}
```

```
class Circle extends Shape {
```

```
    void area() {
```

```
        System.out.println("Area of Circle = " + (3.14 * 7 * 7));
```

```
    }
```

```
}
```

```
public class ShapeDemo {
```

```
    public static void main(String args[]) {
```

```
Shape t = new Triangle();
```

```
Shape r = new Rectangle();
```

```
Shape c = new Circle();
```

```
t.area();
```

```
r.area();
```

```
c.area();
```

```
}
```

```
}
```

Output

Area of Triangle = 25.0

Area of Rectangle = 50

Area of Circle = 153.86

Experiment No. 21

Aim

Write a Java program to demonstrate the implementation of **multiple inheritance using interfaces**.

Theory

Java does not support multiple inheritance using classes because it can create ambiguity. However, multiple inheritance can be achieved using **interfaces**. A class can implement more than one interface.

Algorithm

1. Create two interfaces.
 2. Declare one method in each interface.
 3. Create a class implementing both interfaces.
 4. Implement both methods.
 5. Create an object and invoke the methods.
-

Program

```
interface Printer {  
    void print();  
}
```

```
interface Scanner {  
    void scan();  
}
```

```
class AllInOne implements Printer, Scanner {
```

```
    public void print() {  
        System.out.println("Printing Document...");  
    }
```

```
    public void scan() {  
        System.out.println("Scanning Document...");  
    }  
}
```

```
public class MultipleInheritance {
```

```
    public static void main(String args[]) {
```

```
        AllInOne obj = new AllInOne();
```

```
        obj.print();
```

```
        obj.scan();
```

```
    }
```

```
}
```

Sample Output

Printing Document...

Scanning Document...

Result

The program successfully demonstrated multiple inheritance using interfaces.

Experiment No. 22

Aim

Write a Java program to demonstrate the use of a **final class**.

Theory

A **final class** cannot be inherited by any other class.

Algorithm

1. Create a final class.
 2. Define a method.
 3. Create an object.
 4. Call the method.
-

Program

```
final class Demo {  
  
    void display() {  
  
        System.out.println("This is a Final Class.");  
  
    }  
}  
  
public class FinalClassDemo {  
  
    public static void main(String args[]) {
```

```
Demo obj = new Demo();

obj.display();

}

}
```

Sample Output

This is a Final Class.

Result

The program successfully demonstrated the use of a final class.

Experiment No. 23

Aim

Write a Java program to develop a **user-defined exception** for "Divide by Zero".

Theory

A user-defined exception is created by extending the Exception class.

Algorithm

1. Create a custom exception class.
 2. Read two numbers.
 3. Check whether the divisor is zero.
 4. Throw the exception.
 5. Catch and display the message.
-

Program

```
import java.util.Scanner;

class DivideByZeroException extends Exception {

    DivideByZeroException(String msg) {
```

```
super(msg);
```

```
}
```

```
}
```

```
public class UserException {
```

```
    public static void main(String args[]) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int a, b;
```

```
        System.out.print("Enter First Number: ");
```

```
        a = sc.nextInt();
```

```
        System.out.print("Enter Second Number: ");
```

```
        b = sc.nextInt();
```

```
        try {
```

```
            if (b == 0)
```

```
                throw new DivideByZeroException("Cannot Divide by Zero");
```

```
            System.out.println("Answer = " + (a / b));
```

```
        }
```

```
        catch (DivideByZeroException e) {
```

```
            System.out.println(e.getMessage());
```

```
        }
```

```
        sc.close();  
  
    }  
}
```

Sample Output

Enter First Number: 20

Enter Second Number: 0

Cannot Divide by Zero

Result

The user-defined exception was successfully demonstrated.

Experiment No. 24

Aim

Write a Java program to demonstrate **multiple try blocks** and **multiple catch blocks**.

Theory

Java allows multiple try-catch statements and multiple catch blocks to handle different exceptions.

Algorithm

1. Write two try blocks.
 2. Create different exceptions.
 3. Catch them using appropriate catch blocks.
 4. Display messages.
-

Program

```
public class MultipleTryCatch {  
  
    public static void main(String args[]) {
```

```
try {  
  
    int a = 10 / 0;  
  
}  
  
catch (ArithmeticException e) {  
  
    System.out.println("Arithmetic Exception");  
  
}  
  
try {  
  
    int arr[] = new int[5];  
  
    System.out.println(arr[10]);  
  
}  
  
catch (ArrayIndexOutOfBoundsException e) {  
  
    System.out.println("Array Index Exception");  
  
}  
  
}
```

Sample Output

Arithmetic Exception

Array Index Exception

Experiment No. 25

Aim

Develop a Banking Application that throws **"Not Sufficient Fund"** when the withdrawal amount exceeds the available balance.

Theory

Exceptions can be used to prevent invalid banking operations such as withdrawing more money than the available balance.

Algorithm

1. Initialize balance to ₹1000.
 2. Deposit ₹1000.
 3. Withdraw ₹400.
 4. Withdraw ₹300.
 5. Attempt to withdraw ₹500.
 6. Throw an exception if the balance is insufficient.
-

Program

```
class InsufficientFundException extends Exception {

    InsufficientFundException(String msg) {

        super(msg);

    }

}

class Bank {

    double balance = 1000;

    void withdraw(double amount) throws InsufficientFundException {

        if (amount > balance)
```

```
        throw new InsufficientFundException("Not Sufficient Fund");
```

```
        balance = balance - amount;
```

```
        System.out.println("Withdraw : " + amount);
```

```
        System.out.println("Balance : " + balance);
```

```
    }
```

```
}
```

```
public class BankingApplication {
```

```
    public static void main(String args[]) {
```

```
        Bank b = new Bank();
```

```
        try {
```

```
            b.withdraw(400);
```

```
            b.withdraw(300);
```

```
            b.withdraw(500);
```

```
        }
```

```
        catch (InsufficientFundException e) {
```

```
            System.out.println(e.getMessage());
```

```
        }
```

```
}  
}
```

Sample Output

Withdraw : 400.0

Balance : 600.0

Withdraw : 300.0

Balance : 300.0

Not Sufficient Fund

Experiment No. 26

Aim

Write a Java program that executes two threads. One thread displays "**Thread1**" every **2000 milliseconds**, and the other displays "**Thread2**" every **4000 milliseconds**.

Theory

Threads allow multiple tasks to execute simultaneously. By extending the Thread class, independent threads can be created.

Algorithm

1. Create two classes extending Thread.
 2. Override the run() method.
 3. Use sleep() to delay execution.
 4. Start both threads.
-

Program

```
class Thread1 extends Thread {
```

```
    public void run() {
```

```
        try {
```

```
        for (int i = 1; i <= 5; i++) {

            System.out.println("Thread1");

            Thread.sleep(2000);

        }

    }

    catch (InterruptedException e) {

        System.out.println(e);

    }

}

}
```

```
class Thread2 extends Thread {

    public void run() {

        try {

            for (int i = 1; i <= 5; i++) {

                System.out.println("Thread2");

                Thread.sleep(4000);

            }

        }

    }

}
```

```

    }

    catch (InterruptedException e) {

        System.out.println(e);

    }

}

}

}

public class ThreadDemo {

    public static void main(String args[]) {

        Thread1 t1 = new Thread1();

        Thread2 t2 = new Thread2();

        t1.start();

        t2.start();

    }

}

```

Sample Output

```

Thread1
Thread2
Thread1
Thread1
Thread2
Thread1

```

Thread1

Thread2

Thread2

Thread2

(The exact output order may vary because the threads execute concurrently.)

Experiment No. 27

Aim

Write a Java program that executes two threads. One thread prints the **even numbers** and the other thread prints the **odd numbers** from **1 to 50**.

Theory

Multithreading allows multiple tasks to execute concurrently. In this program, two threads are created by extending the Thread class. One thread prints even numbers, while the other prints odd numbers.

Algorithm

1. Create a class EvenThread extending Thread.
2. Override the run() method to print even numbers from 1 to 50.
3. Create another class OddThread extending Thread.
4. Override the run() method to print odd numbers from 1 to 50.
5. Create objects of both classes.
6. Start both threads.

Program

```
class EvenThread extends Thread {  
  
    public void run() {  
  
        System.out.println("Even Numbers:");  
  
        for (int i = 2; i <= 50; i = i + 2) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

```
}
```

```
class OddThread extends Thread {  
  
    public void run() {  
  
        System.out.println("\nOdd Numbers:");  
  
        for (int i = 1; i <= 50; i = i + 2) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

```
public class EvenOddThread {  
  
    public static void main(String args[]) {  
  
        EvenThread t1 = new EvenThread();  
        OddThread t2 = new OddThread();  
  
        t1.start();  
        t2.start();  
    }  
}
```

Sample Output

Even Numbers:

2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50

Odd Numbers:

1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49

Note: Since threads execute concurrently, the output order may vary.

Result

The program successfully demonstrated multithreading by printing even and odd numbers using two separate threads.

Experiment No. 28

Aim

Write a Java program to demonstrate **thread synchronization** when multiple threads update a common variable.

Theory

Synchronization ensures that only one thread accesses a shared resource at a time, preventing data inconsistency.

Algorithm

1. Create a shared class containing a common variable.
 2. Create a synchronized method to update the variable.
 3. Create two thread classes.
 4. Both threads call the synchronized method.
 5. Display the updated value.
-

Program

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
  
        count++;  
  
        System.out.println(Thread.currentThread().getName()  
            + " Count = " + count);  
  
    }  
}  
  
class MyThread extends Thread {  
  
    Counter c;  
  
    MyThread(Counter c, String name) {
```

```
        super(name);

        this.c = c;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            c.increment();
        }
    }
}

public class SynchronizationDemo {

    public static void main(String args[]) {

        Counter c = new Counter();

        MyThread t1 = new MyThread(c, "Thread-1");
        MyThread t2 = new MyThread(c, "Thread-2");

        t1.start();
        t2.start();
    }
}
```

Sample Output

```
Thread-1 Count = 1
Thread-2 Count = 2
Thread-1 Count = 3
Thread-2 Count = 4
Thread-1 Count = 5
Thread-2 Count = 6
Thread-1 Count = 7
Thread-2 Count = 8
Thread-1 Count = 9
Thread-2 Count = 10
```

Result

The program successfully demonstrated synchronization while updating a common variable using multiple threads.

Experiment No. 29

Aim

Write a Java program to perform **Create, Write, Modify, and Read** operations on a text file.

Theory

Java provides the java.io package for file handling. Using FileWriter, BufferedReader, and FileReader, text files can be created, modified, written to, and read.

Algorithm

1. Create a text file.
 2. Write data to the file.
 3. Append additional data.
 4. Read the file contents.
 5. Display the contents.
-

Program

```
import java.io.*;
public class FileOperation {

    public static void main(String args[]) throws Exception {

        // Create and Write
        FileWriter fw = new FileWriter("student.txt");

        fw.write("Payal Vasoya\n");
        fw.write("BCA Semester 3\n");

        fw.close();

        // Modify (Append)
        FileWriter fw1 = new FileWriter("student.txt", true);

        fw1.write("Object Oriented Programming with Java\n");

        fw1.close();
```

```
// Read
BufferedReader br = new BufferedReader(new FileReader("student.txt"));

String line;

System.out.println("File Contents:");

while ((line = br.readLine()) != null) {

    System.out.println(line);

}

br.close();
}
```

Sample Output

File Contents:

Payal Vasoya

BCA Semester 3

Object Oriented Programming with Java